

 HANDOFF.md — เอกสารส่งต่องาน (สำหรับ AI ตัวถัดไป)

อ่านเอกสารนี้เป็นอันดับแรก ก่อนที่จะเริ่มทำงานใด ๆ เอกสารนี้บอกว่าคุณอยู่จุดไหน + ต้องทำอะไรต่อ

อ่านประกอบด้วย:

- 1. `PLAN.md` — แผนแม่บทและ Checklist
- 2. `PROJECT_ANALYSIS.md` — บริบทธุรกิจ
- 3. `HANDOFF.md` (ไฟล์นี้) — สถานะปัจจุบัน

 1. สถานะปัจจุบัน (Current Status) ณ วันที่ 2026-05-18

 สิ่งที่ทำเสร็จแล้ว (Completed)

Phase	งาน	ไฟล์
0	วิเคราะห์ข้อเสนอ	<code>PROJECT_ANALYSIS.md</code>
1	แผนงาน + Checklist	<code>PLAN.md</code>
1	JSON ฐานข้อมูล 6 ตาราง พร้อม sample + UUID	<code>data/*.json</code>
1	Config ทั้งหมด	<code>config/app.php</code> , <code>config/routes.php</code> , <code>config/line.php</code> , <code>config/slip2go.php</code>
2	Core Framework ครบทุกตัว	<code>app/Core/Database.php</code> , <code>Session.php</code> , <code>Request.php</code> , <code>View.php</code> , <code>Controller.php</code> , <code>Router.php</code>
3	MySQL Migration 	<code>database/infinityfree_schema.sql</code> , <code>database/migrate_json_to_mysql.php</code>
3	MySQL Database Core 	<code>app/Core/MysqlDatabase.php</code> , <code>app/Core/DatabaseInterface.php</code>
3	Repository Layer (MySQL Ready) 	<code>app/Repositories/BaseRepository.php</code> (รองรับ JSON/MySQL), ทุก Repository อัปเดตแล้ว

 สิ่งที่ต้องทำต่อ (Next Steps — เรียงตามลำดับ)

 ลำดับที่ 1: Models (Phase 3)

สร้าง 6 ไฟล์ ใน `app/Models/`:

- `User.php`, `Service.php`, `Staff.php`, `Booking.php`, `Payment.php`, `Review.php`

แต่ละ Model = คลาส POPO (Plain Old PHP Object) เก็บโครงสร้างข้อมูล + constants **ไม่มี logic** ฐานข้อมูล (ฐานข้อมูลอยู่ใน Repository)

ตัวอย่าง template:

```
<?php
namespace App\Models;

class Booking
{
    // สถานะที่เป็นไปได้ / Possible statuses
    public const STATUS_PENDING      = 'pending';
    public const STATUS_CONFIRMED    = 'confirmed';
    public const STATUS_IN_SERVICE   = 'in_service';
    public const STATUS_COMPLETED    = 'completed';
    public const STATUS_CANCELLED    = 'cancelled';

    public int $id;
    public string $uuid;
    public string $booking_code;
    public int $member_id;
    public int $service_id;
    public int $staff_id;
    public string $booking_date;    // YYYY-MM-DD
    public string $start_time;      // HH:MM
    public string $end_time;
    public float $total_price;
    public string $status;
    public ?string $note;
    public string $created_at;
    public string $updated_at;

    public static function fromArray(array $row): self { /* mapping */ }
    public function toArray(): array { /* serialize */ }
}
```

✅ ลำดับที่ 2: Repositories (Phase 3) — DONE

✅ เสร็จสมบูรณ์แล้ว — ทุก Repository รองรับทั้ง JSON และ MySQL

- BaseRepository.php — ปรับปรุงให้รองรับ driver config (json/mysql)
- UserRepository.php, ServiceRepository.php, StaffRepository.php
- BookingRepository.php, PaymentRepository.php, ReviewRepository.php, SettingRepository.php

การทำงาน:

- อ่านค่า config/database.php['driver']
- ถ้าเป็น 'mysql' → ใช้ MysqlDatabase class
- ถ้าเป็น 'json' → ใช้ Database (JSON) class
- Repository ทั้งหมดใช้ DatabaseInterface จึงสลับ backend ได้โดยไม่กระทบ Service Layer

● ลำดับที่ 3: Services (Phase 4)

สร้างใน `app/Services/`:

- **AuthService.php**: register, login (verify hash), logout, hashPassword
- **BookingService.php**: createBooking, checkTimeConflict, cancelBooking, generateBookingCode (เช่น WK20260518-006)
- **PaymentService.php**: createPayment, attachSlip, verifyPayment (เรียก Slip2GoService), approveManually
- **ReportService.php**: revenueByDay, revenueByMonth, revenueByYear, topServices
- **LineNotifyService.php**: pushMessage(userId, text), broadcast(text) — ถ้า `config.enabled=false` ให้ log แทน
- **Slip2GoService.php**: verifySlip(\$imagePath, \$expectedAmount) — ใช้ curl POST ไป endpoint จาก config

● ลำดับที่ 4: REST API (Phase 5a) ★ ลำดับสำคัญ

เนื่องจาก UI จะเรียก API → ทำ API ก่อน Controllers/Views

สร้าง `api/index.php` ทำหน้าที่ router + helper:

```
<?php
// ตั้งค่า autoload + CORS + JSON response header
require __DIR__ . '/../app/Core/autoload.php'; // ต้องสร้าง

header('Content-Type: application/json; charset=utf-8');
header('Access-Control-Allow-Origin: *');
header('Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS');
header('Access-Control-Allow-Headers: Content-Type, Authorization');

if ($_SERVER['REQUEST_METHOD'] === 'OPTIONS') exit;

// อ่าน path เช่น /api/v1/bookings/3
$uri = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
// ตัด /api/ ออก → เหลือ v1/bookings/3
$path = preg_replace('#^.*?/api/#', '', $uri);
$parts = explode('/', trim($path, '/')); // ['v1', 'bookings', '3']

$version = $parts[0] ?? 'v1';
$resource = $parts[1] ?? null;

// route ไปไฟล์ที่ตรงกับ resource
$file = __DIR__ . "/{$version}/{$resource}.php";
if (file_exists($file)) {
    require $file;
} else {
    http_response_code(404);
    echo json_encode(['success' => false, 'message' => 'Not found']);
}
```

แต่ละไฟล์ `api/v1/*.php` รับ method + parameters แล้วเรียก Service:

```

<?php
// api/v1/services.php
use App\Services\AuthService;
use App\Repositories\ServiceRepository;

$method = $_SERVER['REQUEST_METHOD'];
$repo = new ServiceRepository();

switch ($method) {
    case 'GET':
        // ถ้ามี id ใน URL ($parts[2]) → ดึงรายตัว
        // else → ดึงทั้งหมด
        if (isset($parts[2])) {
            $row = $repo->find((int) $parts[2]);
            echo json_encode(['success' => (bool)$row, 'data' => $row]);
        } else {
            echo json_encode(['success' => true, 'data' => $repo->all()]);
        }
        break;
    case 'POST': /* create */ break;
    case 'PUT': /* update */ break;
    case 'DELETE': /* delete */ break;
}

```

● ลำดับที่ 5: Web Controllers (Phase 5b)

สร้างใน `app/Controllers/`:

- `HomeController.php` — `index()`, `services()`, `about()`
- `AuthController.php` — `showLogin()`, `showRegister()`, `logout()` (Web เท่านั้น)
- `MemberController.php` — `dashboard()`, `bookings()`, `profile()`
- `BookingController.php` — `create()`, `show($id)`
- `StaffController.php` — `dashboard()`, `bookings()`
- `AdminController.php` — `dashboard()`, `users()`, `services()`, `staff()`, `bookings()`, `payments()`, `report()`

ห้าม: action ที่เปลี่ยน state (เช่น login, store, update) — ให้ frontend เรียก API แทน

● ลำดับที่ 6: Views (Phase 5b) — UI สไตล์ Lovable

สร้างใน `app/Views/`:

Layout หลัก (`layouts/main.php`):

```

<!DOCTYPE html>
<html lang="th">
    <head>
        <meta charset="UTF-8" />
        <title><?= $title ?? 'Wikanda Hair Salon' ?></title>
        <link

```

```

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
    rel="stylesheet"
/>
<link
    href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700&display=swap"
    rel="stylesheet"
/>
<link
    href="/Wikanda_Hair_Salon/public/assets/css/style.css"
    rel="stylesheet"
/>
</head>
<body>
    <?php include __DIR__ . '/../partials/navbar.php'; ?>
    <main><?= $content ?></main>
    <?php include __DIR__ . '/../partials/footer.php'; ?>
</body>
</html>

```

Lovable Style Guidelines:

- ฟอนต์: Plus Jakarta Sans หรือ Inter
- สี:
 - พื้นหลัง: #FFF8F3 (ครีม) หรือ #FFFFFF
 - Primary gradient: linear-gradient(135deg, #FFB6D5 0%, #C8A8FF 100%)
 - Accent ชมพู: #FF6B9D
 - ม่วง: #9B6BFF
- Border-radius: 16-24px (card), 12px (input/button)
- Shadow: 0 8px 30px rgba(0, 0, 0, 0.04) (soft)
- Glassmorphism: backdrop-filter: blur(20px) + background: rgba(255,255,255,0.7)
- ปุ่ม: gradient พร้อม hover lift transform: translateY(-2px)
- Animation: transition: all 0.3s cubic-bezier(0.4, 0, 0.2, 1)

● ลำดับที่ 7: Public Entry (Phase 2 ต่อ)

สร้าง:

- **public/index.php** — Front Controller

```

<?php
require_once __DIR__ . '/../app/Core/autoload.php';

use App\Core\Router;
use App\Core\Session;

Session::start();
date_default_timezone_set('Asia/Bangkok');

```

```
(new Router())->dispatch();
```

- **public/.htaccess** — URL rewrite

```
RewriteEngine On
RewriteCond %{REQUEST_FILENAME} !-d
RewriteCond %{REQUEST_FILENAME} !-f
RewriteRule ^(.*)$ index.php [QSA,L]
```

- **app/Core/autoload.php** — PSR-4 style autoloader

```
<?php
spl_autoload_register(function ($class) {
    // App\Core\Database → app/Core/Database.php
    $prefix = 'App\\';
    $base   = __DIR__ . '/../';
    if (!str_starts_with($class, $prefix)) return;
    $relative = substr($class, strlen($prefix));
    $file = $base . str_replace('\\', '/', $relative) . '.php';
    if (file_exists($file)) require $file;
});
```

- **public/assets/css/style.css** — Lovable style
- **public/assets/js/main.js** — vanilla JS เรียก fetch(/api/v1/...)

● ลำดับที่ 8: README.md

อธิบายวิธีติดตั้งบน XAMPP, account ทดสอบ, การเข้าถึง URL

🔧 2. กฎที่ต้องเคารพ (Rules to Follow)

กฎการเขียนโค้ด (จาก PLAN.md ข้อ 4)

1. ✅ ทุกไฟล์ + ทุกฟังก์ชัน มี **comment ไทย-อังกฤษ**
2. ✅ ใช้ **type hint** และ **return type** เสมอ
3. ✅ 1 ฟังก์ชัน = 1 หน้าที่ (Single Responsibility)
4. ✅ Controller บาง — logic อยู่ใน Service
5. ✅ Repository ปิดบังการอ่าน/เขียน — Service ไม่รู้ว่าใช้ JSON หรือ MySQL
6. ✅ ทุกแถวมี **id** (auto-increment) + **uuid** (auto-gen) เสมอ
7. ✅ ห้ามทำงานบน **Docker** — ใช้ XAMPP เท่านั้น
8. ✅ การตรวจสอบสลิปใช้ **Slip2Go** API
9. ✅ UI สไตล์ **Lovable** (soft, gradient, glassmorphism)

Stack

- PHP 8+ (pure, no framework)
- HTML/CSS/JS + Bootstrap 5
- Storage: JSON files (จะย้ายเป็น MySQL ในอนาคต)
- Notification: LINE Messaging API
- Slip Verify: Slip2Go API

3. โครงสร้างไฟล์ปัจจุบัน (Current Tree)

Wikanda_Hair_Salon/	
├── PLAN.md	✓
├── PROJECT_ANALYSIS.md	✓
├── HANDOFF.md	✓ (ไฟล์นี้)
├── README.md	✗ (ยังไม่สร้าง)
├── api/	✗ ทั้งโฟลเดอร์ยังไม่สร้าง
│ └── v1/	
├── public/	✗ ทั้งโฟลเดอร์ยังไม่สร้าง
│ ├── index.php	
│ ├── .htaccess	
│ └── assets/	
├── app/	
│ ├── Core/	
│ │ ├── Database.php	✓ (มี UUID generator)
│ │ ├── Session.php	✓
│ │ ├── Request.php	✓
│ │ ├── View.php	✓
│ │ ├── Controller.php	✓
│ │ ├── Router.php	✓
│ │ └── autoload.php	✗ ⚠ ต้องสร้างก่อนใช้คลาสได้
│ ├── Models/	✗
│ ├── Repositories/	✗
│ ├── Services/	✗
│ ├── Controllers/	✗
│ ├── Middleware/	✗
│ └── Views/	✗
├── config/	
│ ├── app.php	✓
│ ├── routes.php	✓
│ ├── line.php	✓
│ └── slip2go.php	✓
├── data/	
│ ├── users.json	✓ (6 users)
│ ├── services.json	✓ (7 services)
│ ├── staff.json	✓ (2 staff)
│ ├── bookings.json	✓ (5 bookings)
│ ├── payments.json	✓ (3 payments)
│ └── reviews.json	✓ (2 reviews)

```
└─ storage/
   └─ uploads/slips/
   └─ logs/
```

✗ (สร้างโฟลเดอร์เปล่า)
✗ (สร้างโฟลเดอร์เปล่า)

🔑 4. ข้อมูลทดสอบ (Test Accounts)

รหัสผ่านทั้งหมด: **password123**

Role	Username	Email
Admin	admin	admin@wikanda.local
Owner	owner	owner@wikanda.local
Staff	staff01	staff01@wikanda.local
Staff	staff02	staff02@wikanda.local
Member	member01	member01@example.com
Member	member02	member02@example.com

⚠ **หมายเหตุ:** Hash password ใน **users.json** เป็น placeholder ตอนรันจริงครั้งแรก ให้สร้าง script เล็ก ๆ:

```
echo password_hash('password123', PASSWORD_BCRYPT);
```

แล้วเอาค่าที่ได้ไป replace ใน **users.json** ทุกแถว

🚀 5. เริ่มทำต่ออย่างไร

ขั้นแรก: ตรวจสอบสถานะ

- อ่าน PLAN.md ดู Checklist ข้อ 5
- ดูข้อ "สิ่งที่ต้องทำต่อ" ในไฟล์นี้
- ทำตามลำดับ 1 → 8

ขั้นที่สอง: ทำตามลำดับ

สำคัญ: ทำ ลำดับที่ 1-3 (**Models, Repos, Services**) ก่อน แล้วค่อยทำ **API** เพราะ API ใช้ Service และ Service ใช้ Repository

ขั้นที่สาม: อัปเดต PLAN.md

- ทุกครั้งที่ทำ task เสร็จ ให้เปลี่ยน - [] เป็น - [x] ใน **PLAN.md** ข้อ 5
- ทุกครั้งที่มี decision ใหม่ ให้ log ใน **PLAN.md** ข้อ 9 (Change Log)

ขั้นที่สี่: ทดสอบ

1. Copy โพรเจกต์ไปใต้ `C:\xampp\htdocs\Wikanda_Hair_Salon\`
2. เปิด XAMPP → Start Apache
3. เข้า `http://localhost/Wikanda_Hair_Salon/public/`
4. ทดสอบ API: `http://localhost/Wikanda_Hair_Salon/api/v1/services`

🗨 6. สิ่งที่ใช้สั่งเพิ่มตลอดทาง (Decisions Log)

จากการสนทนา ผู้ใช้ได้สั่งการเพิ่มดังนี้:

1. ✅ ใช้ **XAMPP** ไม่ใช่ Docker
2. ✅ ตรวจสอบด้วย **Slip2Go API**
3. ✅ UI สไตล์ **Lovable** (modern, soft, gradient, glassmorphism)
4. ✅ มีโฟลเดอร์ `/api` เป็น PHP คั่น JSON (ใช้ JSON storage ก่อน)
5. ✅ JSON structure ต้องตรงกับ SQL table (เพื่อ migrate ในอนาคต)
6. ✅ ทุกตารางต้องมี **UUID** ทุกแถว (auto-generated)
7. ✅ Comment ทุกที่ **ไทย-อังกฤษ**
8. ✅ โค้ดต้อง **basic** เด็กจบใหม่อ่านได้ ไม่อวดเทคนิค

⚠ 7. ข้อควรระวัง (Pitfalls)

1. **Namespace + Autoload:** ก่อนรันต้องสร้าง `app/Core/autoload.php` ก่อน ไม่งั้น use `App\Core\Database` จะ error
2. **JSON Race Condition:** `Database.php` ใช้ `flock()` แล้ว แต่ถ้า traffic เยอะ อาจมี edge case → migrate เป็น MySQL จะปลอดภัยกว่า
3. **Password Hash:** ใน sample data เป็น placeholder ไม่ใช่ hash จริง — ต้อง re-hash ก่อนใช้
4. **CORS:** API ต้องตั้ง CORS header ถ้าเรียกจาก origin อื่น
5. **path บน Windows:** XAMPP บน Windows ใช้ backslash บางที — เวลาเขียน path ใช้ `__DIR__` + forward slash จะปลอดภัย
6. **Slip2Go + LINE:** ทั้งสอง config มี `enabled` flag — เริ่มต้นปิดไว้ ตอน dev จะ log แทนการเรียก API จริง

📞 8. ติดต่อ / Context เพิ่มเติม

- เจ้าของโปรเจกต์: นายวันมงคล เกตุพุด (รหัส 66541207002-1)
- อาจารย์ที่ปรึกษา: ผศ. วรการ ใจดี
- กรณีศึกษา: ร้าน Wikanda Hair Salon (เชียงใหม่)
- ระยะเวลา: พ.ย. 2568 – ต.ค. 2569

สรุปสั้น ๆ: Phase 1-2 เสร็จ (โครงสร้าง + Core Framework + JSON Data) → ลำดับถัดไปคือ **Models** → **Repositories** → **Services** → **API** → **Controllers** → **Views**

อย่าลืม update `PLAN.md` ทุกครั้งที่ทำเสร็จ และอ่าน HANDOFF นี้ใหม่ถ้าสับสน